

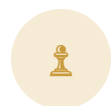
MSC AI & DATA SCIENCE DISSERTATION · CITY COLLEGE, UNIVERSITY OF YORK

Adaptive Chess Puzzle Advisor

An adaptive, weakness-targeted chess puzzle recommender built on a Thompson Sampling bandit, Stockfish-verified puzzle mining, and Glicko-2 difficulty calibration.

OVERVIEW

What the system does



Analyse

Pulls a player's Chess.com game history and estimates which of 23 tactical categories they're weakest in.



Recommend, adaptively

A Thompson Sampling multi-armed bandit selects which category to practise next, updating in real time from every solve/fail.



Mine & calibrate

Puzzles come from a 6M-row Lichess database plus puzzles mined directly from the player's own games; two offline models (Glicko-2, a trained classifier) extend the calibration further.

23

WEAKNESS CATEGORIES

6M+

LICHESS PUZZLES

128

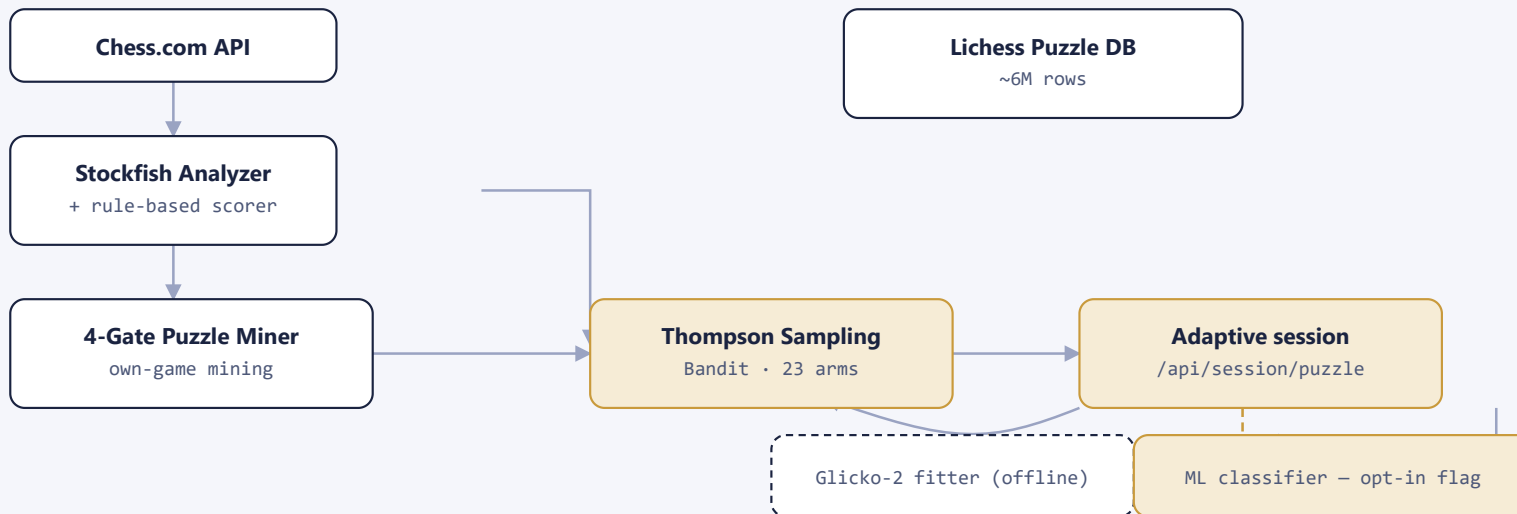
PASSING TESTS

4

REAL USERS SO FAR

SYSTEM DESIGN

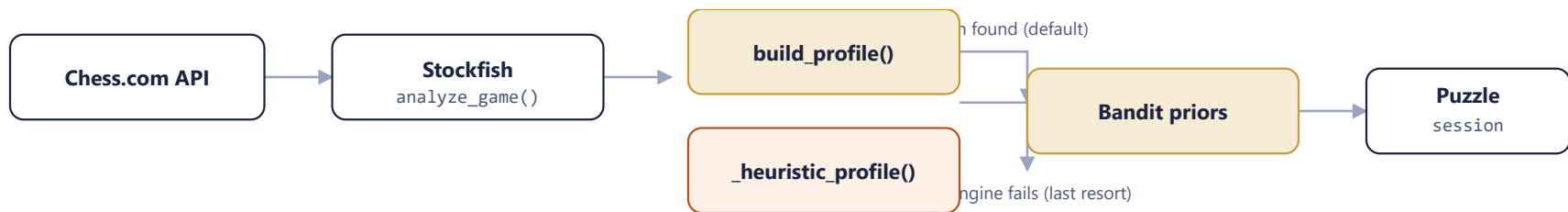
Architecture at a glance



Solid = live in the app today — including the ML classifier, reachable via the `WEAKNESS_MODEL=m1` opt-in flag. Dashed gold = built and validated, not yet wired into the serving path (Glicko-2 fitter only).

PIPELINE 1

Player analysis — from games to a weakness profile



- › Every player move is evaluated before and after with Stockfish — a ≥ 50 cp drop is logged as an error, with severity and now also a tactical **category**.
- › `build_profile()` converts error patterns into per-category weakness scores via hand-tuned thresholds (not learned — see slide 5).
- › A crude engine-free fallback exists for when Stockfish can't be located — more on that on slide 15.

PIPELINE 1, CONTINUED

The rule-based weakness scorer

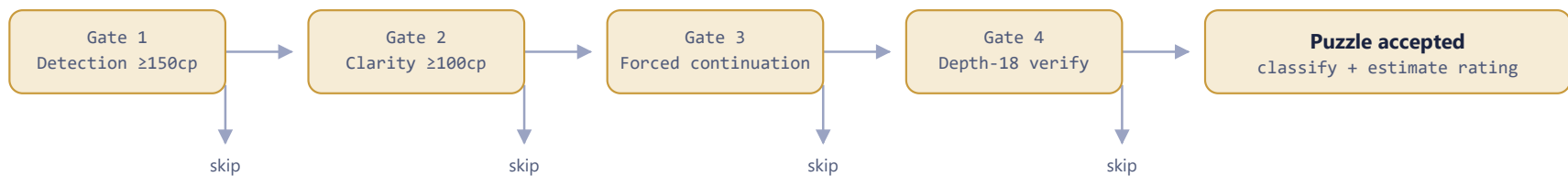
SIGNAL	RULE	EFFECT
Middlegame error rate > 0.5	<code>boost = min(0.25, rate × 0.3)</code>	Fork / Pin / Skewer / Sacrifice / Deflection / Hanging Piece / King Safety / Discovered Attack each +boost
Endgame error rate > 0.25	<code>boost = min(0.30, rate × 0.4)</code>	All 6 endgame categories +boost
Blunders/game ≥ 2	fixed +0.25	Hanging Piece, Fork, Pin
Blunders/game ≥ 1	fixed +0.15	Hanging Piece, Fork
Accuracy estimate < 70	fixed +0.10	Fork, Pin, Skewer, Discovered Attack

Why this matters

All 23 categories start at 0.5 and move via hard-coded if/else thresholds — not learned from data. This is exactly the gap the trained classifier on slide 12–13 was built to close.

PIPELINE 2

Mining puzzles from the player's own games



150cp

Minimum eval-drop to count as a real blunder (`PUZZLE_THRESHOLD`)

100cp

PV1 must beat PV2 by this much — rejects dual-solution puzzles

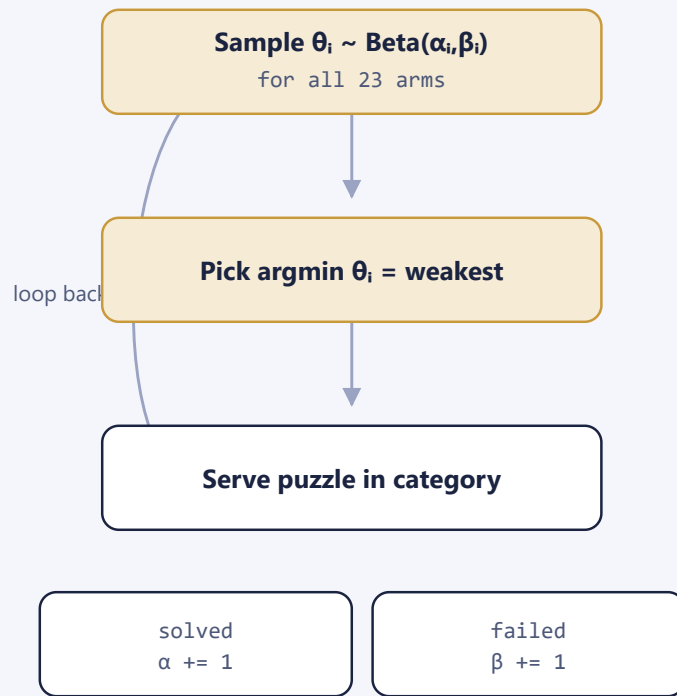
Depth 18

Final re-verification pass before the puzzle is accepted

An earlier heuristic-only (no-engine) mode was tried and **removed** — it produced ~40% false positives.

PIPELINE 3

The Thompson Sampling recommendation loop



Serving cascade order:

1. 🚩 User-mined puzzle, target category, unseen
2. 🚩 Any unseen mined puzzle (40% of the time)
3. 🚩 Lichess pool, target category, rating window
4. 🚩 Lichess pool, widened rating window
5. 🚩 Lichess pool, any category
6. 🚩 Last resort: allow a repeat, flagged explicitly

THE MATHEMATICS

Thompson Sampling — why Beta(α , β)?

Each category is a **Bernoulli process**: every attempt is a solve (1) or fail (0) with unknown true probability p_i .

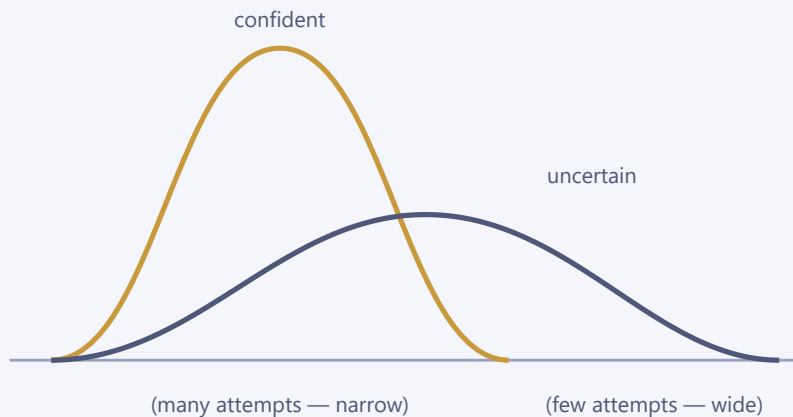
Beta is the **conjugate prior** for Bernoulli — the posterior after new evidence is exactly another Beta, in closed form:

```
if solved: arm.alpha += 1
else: arm.beta += 1
```

QUANTITY	MEANING
$\alpha - 1$	effective solves seen (real + prior)
$\beta - 1$	effective fails seen
$\alpha + \beta$	total confidence — bigger = narrower posterior
mean	$\alpha / (\alpha + \beta)$ — best point estimate
variance	$\alpha\beta / [(\alpha + \beta)^2(\alpha + \beta + 1)]$ — shrinks with evidence

THE MATHEMATICS, CONTINUED

Sampling — not averaging — is what makes exploration free



Worked example

"Fork" attempted 8×, solved 3× → $\alpha=4$, $\beta=6$, mean=0.4.
A sample might land at 0.28 (looks weaker) or 0.55 (looks fine) — both are legitimate draws. The randomness is the mechanism, not noise.

The one deliberate inversion

`select_one()` picks the **lowest** sampled θ , not the highest — θ measures proficiency here, and the goal is to practise weaknesses, not strengths.

OFFLINE RESEARCH COMPONENT

Glicko-2 — beyond a single Elo number

SYMBOL	NAME	MEANING
r / rating	Rating	same interpretation as Elo — higher is stronger / harder
RD / rd	Rating Deviation	the system's <i>uncertainty</i> — $\text{rating} \pm 2 \times \text{RD} \approx 95\%$ confidence interval
σ / volatility	Volatility	how erratically true skill seems to move — real change vs normal noise

A bare Elo number can't tell "we're very sure" from "we just guessed." A new puzzle or player starts with high RD; it shrinks as real evidence accumulates.

OFFLINE RESEARCH COMPONENT

Update mechanics & validation

1. $\gamma g(\phi_j)$ — discount by the opponent's own uncertainty
2. $\gamma E(\mu, \mu_j, \phi_j)$ — expected outcome (logistic, like Elo)
3. γv — variance of the rating change
4. $\gamma \Delta$ — actual vs expected outcome, scaled by v
5. Solve for new σ' via the Illinois root-finder
6. Update ϕ' , μ' ; convert back to (rating, RD)

Matches Glickman's own published example

1500 / RD200 / vol0.06, three games → **1464.06 / 151.52 / 0.05999**, matched to tested tolerance.

105

EVENTS FITTED

3

REAL PLAYERS

61

PUZZLES FITTED

OFFLINE RESEARCH COMPONENT

Trained weakness classifier — `build_profile_ml()`

- › Drop-in alternative to the rule-based scorer — same signature, same output shape, only `weakness_scores` differs.
- › **30 input features**: 7 coarse summary stats + 23 per-category observed error rates (now measurable thanks to the new `MoveError.category` field).
- › `RandomForestRegressor` predicts all 23 category weaknesses at once.
- › Trained on **simulated players** — only 4 real players have any mined-puzzle history, far too few for a statistically meaningful held-out-player split.
- › Synthetic data generated via a **Dirichlet-multinomial** process: true-weak categories get boosted concentration, so observed error counts are realistically noisy, not hand-tuned to look good.

OFFLINE RESEARCH COMPONENT

Held-out-player cross-validation (300 synthetic players)

FOLD	MODEL MAE	BASELINE MAE	TOP WEAKNESS IN PREDICTED TOP-3
0	0.1645	0.1674	30%
1	0.1628	0.1649	30%
2	0.1638	0.1674	27%
3	0.1642	0.1677	38%
4	0.1686	0.1710	40%
Mean	0.1648	0.1677	33%

Beats baseline

In every single fold, consistently

~2.5× random

33% top-3 hit rate vs ~13% chance

Honest caveat

Validates methodology, not yet real-world accuracy

THE HONEST ANSWER

What's genuinely AI-driven here

COMPONENT	CATEGORY	LEARNED FROM DATA?
Thompson Sampling bandit	Bayesian reinforcement learning	Yes — live, online
RandomForest classifier	Supervised ML	Yes — trained, reachable opt-in (<code>WEAKNESS_MODEL=m1</code>), not the default
Glicko-2 fitter	Bayesian statistical estimation	Yes — data volume still small
Stockfish (3rd-party)	Classical search AI	Yes, but not this project's contribution
<code>_classify_tactic()</code>	Rule-based / symbolic	No — hand-coded
Rule-based scorer	Hand-tuned thresholds	No

The strongest claim: the live, online **Thompson Sampling bandit** — a textbook Bayesian exploration/exploitation system, updating from every real puzzle attempt.

Known limitations & discovered bugs

✓ Fixed — the real scorer now actually runs

`analysis_start()` used to look for Stockfish via `shutil.which("stockfish")`, which never checked the bundled binary — the app silently fell back to the crude heuristic scorer almost every time. Now calls `find_stockfish()` instead, the same resolver puzzle mining already used. `build_profile()` is the default path on every successful analysis.

✓ Fixed — all three weakness-scoring paths now wired together

Rule-based `build_profile()` is the default (per the fix above). The trained ML classifier `build_profile_ml()` is now reachable too, via a new `WEAKNESS_MODEL=ml` opt-in flag — deliberately not the default, since it's only validated on simulated players so far. Heuristic remains the last-resort fallback.

✓ Found & fixed live — a second bug the first fix exposed

Fixing the scorer meant Stockfish ran against real games for the first time. That immediately surfaced `multipv=1` making `engine.analyse()` return a list, not a dict — crashing every move analysis. Caught in a live smoke test, fixed by dropping the unnecessary kwarg. Zero test coverage before this, since the unit tests are deliberately engine-free.

Still open: stale documentation & two difficulty-tier schemes

README and eval CLI docstrings quote outdated tuning constants; the Lichess pool and mined puzzles use two different tier boundaries. Cosmetic, not yet cleaned up — the honest remaining gap on this list.

EVIDENCE, NOT ASSERTION

Validation summary

128

TESTS, 8 FILES, OFFLINE



GLICKO-2 MATCHES PUBLISHED
PAPER



BANDIT FAVOURS WEAKER ARM
(SEEDED TEST)

5/5

CV FOLDS BEAT BASELINE

- › Every claim in this deck is traceable to a specific file, function, or test in the repository.
- › Where the code contradicts itself or the docs, it's called out explicitly (slide 15) rather than smoothed over.

CHEAT SHEET

Numbers worth having ready

23

WEAKNESS CATEGORIES (BANDIT ARMS)

6M+

LICHESS PUZZLES, FILTERED & RE-CATEGORISED

128

PASSING TESTS, 8 FILES, ALL OFFLINE

4

REAL PLAYERS PROFILED SO FAR

300

SYNTHETIC PLAYERS, 5-FOLD CV

33%

TOP-3 WEAKNESS HIT RATE (~2.5× RANDOM)

10

A + B PRIOR BUDGET PER CATEGORY (BANDIT)

4

STOCKFISH VERIFICATION GATES PER MINED PUZZLE

If a number gets asked for and isn't here, it's in `dissertation_documentation.md` \$5-\$7 with the exact line of code it came from.

PREPARED ANSWERS

Anticipated questions — cheat sheet

IF ASKED...	SHORT ANSWER
Is this reinforcement learning?	Yes — a Bayesian multi-armed bandit (Thompson Sampling). Not deep RL: no neural policy network, no PPO/DQN.
How do you know the bandit actually works?	Unit-tested directly: priors seed correctly, <code>update()</code> moves α/β the right way, <code>select_one()</code> statistically favours the weaker arm under a seeded RNG.
Where does puzzle quality assurance come from?	Two independent layers: the 4-gate mining pipeline rejects bad extractions before saving; a separate 5-dimension evaluator re-scores whatever is kept.
What's the single biggest gap right now?	Data volume, not methodology — only 4 real players so far. The Stockfish-detection bug that used to compound this (slide 15) is fixed.
Why Thompson Sampling, not UCB or epsilon-greedy?	No separate tunable exploration parameter — posterior width does the exploring automatically, and it matches UCB-style performance for Bernoulli rewards.
What would you do with another month?	Collect more real data, run a real A/B of <code>build_profile_m1()</code> vs the rule-based scorer, wire the Glicko-2 fitter into serving, clean up stale docs.

IMPLEMENTATION

Technology stack

LAYER	TECHNOLOGY
Core language	Python 3.13
Chess logic / engine	python-chess, Stockfish 18
Bandit / recommender	Custom Thompson Sampling (numpy.random.beta)
Difficulty rating	Custom from-scratch Glicko-2
Weakness classifier	scikit-learn RandomForestRegressor
Data processing	pandas, pyarrow (Parquet)
Backend / frontend	Flask + flask-cors · vanilla JS SPA, same origin
Testing	pytest — 128 tests, fully offline

ROADMAP

Priorities for the next iteration

1. ➤Collect more real puzzle-session data (currently 4 players, 105 events) so the Glicko-2 fitter and ML classifier have enough evidence to beat the rule-based scorer
2. ➤Run a real A/B comparison of `build_profile_ml()` against `build_profile()` — both are now genuinely reachable in the live app, so this is a data-collection problem, not an engineering one
3. ➤Load the Glicko-2 fitter's output into the live serving path the same way the ML classifier now is
4. ➤Reconcile the two difficulty-tier schemes and refresh stale tuning-constant docs

What each component stands on

COMPONENT	GROUNDING SOURCE	WHAT IT JUSTIFIES
Thompson Sampling bandit <code>recommender/bandit.py</code>	Thompson (1933); Russo <i>et al.</i> (2018)	The posterior-sampling selection rule and its regret guarantees
Choice of TS over UCB / ϵ-greedy	Auer <i>et al.</i> (2002); Chapelle & Li (2011)	Empirical case for TS under Bernoulli rewards, with no tuning parameter
Per-skill mastery posture <code>player_profiler.py</code>	Corbett & Anderson (1995)	Bayesian per-skill belief updated from binary evidence — the ITS template
Glicko-2 fitter <code>difficulty_fitter.py</code>	Glickman (1999, 2012)	The algorithm itself <i>and</i> the worked example used as the test fixture
RandomForest classifier <code>ml_weakness_model.py</code>	Breiman (2001); Pedregosa <i>et al.</i> (2011)	Ensemble regression and the cross-validation protocol
Elo $\pm$ 300 serving band	Vygotsky (1978); Ericsson <i>et al.</i> (1993)	Zone of proximal development; why practice must target current deficiency
Engine ground truth	Nasu (2018); Stockfish	NNUE evaluation — cheap enough to analyse tens of thousands of positions

Every design decision in this project traces to a primary source — including the one place the implementation deliberately *departs* from the literature (arg min, not arg max — slide 9).

References — algorithms & methods

#	REFERENCE
1	Thompson, W. R. (1933). On the likelihood that one unknown probability exceeds another in view of the evidence of two samples. <i>Biometrika</i> , 25(3–4), 285–294.
2	Russo, D. J., Van Roy, B., Kazerouni, A., Osband, I., & Wen, Z. (2018). A tutorial on Thompson sampling. <i>Foundations and Trends in Machine Learning</i> , 11(1), 1–96.
3	Auer, P., Cesa-Bianchi, N., & Fischer, P. (2002). Finite-time analysis of the multiarmed bandit problem. <i>Machine Learning</i> , 47(2–3), 235–256.
4	Chapelle, O., & Li, L. (2011). An empirical evaluation of Thompson sampling. <i>Advances in Neural Information Processing Systems 24</i> , 2249–2257.
5	Sutton, R. S., & Barto, A. G. (2018). <i>Reinforcement Learning: An Introduction</i> (2nd ed.). MIT Press.
6	Glickman, M. E. (1999). Parameter estimation in large dynamic paired comparison experiments. <i>JRSS: Series C (Applied Statistics)</i> , 48(3), 377–394.
7	Glickman, M. E. (2012). <i>Example of the Glicko-2 system</i> [Technical note]. Boston University.
8	Elo, A. E. (1978). <i>The Rating of Chessplayers, Past and Present</i> . Arco Publishing.
9	Breiman, L. (2001). Random forests. <i>Machine Learning</i> , 45(1), 5–32.
10	Silver, D., Hubert, T., Schrittwieser, J., <i>et al.</i> (2018). A general reinforcement learning algorithm that masters chess, shogi and Go through self-play. <i>Science</i> , 362(6419), 1140–1144.

References — data, tools & pedagogy

#	REFERENCE
11	Nasu, Y. (2018). <i>㊦㊦㊦: Efficiently updatable neural-network-based evaluation functions for computer shogi</i> . Ziosoft Computer Shogi Club.
12	Romstad, T., Costalba, M., Kiiski, J., Linscott, G., <i>et al.</i> <i>Stockfish: A free and strong UCI chess engine</i> [Software]. stockfishchess.org
13	Fiekas, N. <i>python-chess: A chess library for Python</i> [Software]. python-chess.readthedocs.io
14	Lichess. <i>lichess.org open database — puzzles</i> [Dataset, CC0 1.0]. database.lichess.org/#puzzles
15	Chess.com. <i>Published-Data API documentation</i> . chess.com/news/view/published-data-api
16	Pedregosa, F., Varoquaux, G., Gramfort, A., <i>et al.</i> (2011). Scikit-learn: Machine learning in Python. <i>JMLR</i> , 12, 2825–2830.
17	McKinney, W. (2010). Data structures for statistical computing in Python. <i>Proc. 9th Python in Science Conference</i> , 56–61.
18	Corbett, A. T., & Anderson, J. R. (1995). Knowledge tracing: Modeling the acquisition of procedural knowledge. <i>User Modeling and User-Adapted Interaction</i> , 4(4), 253–278.
19	Vygotsky, L. S. (1978). <i>Mind in Society: The Development of Higher Psychological Processes</i> . Harvard University Press.
20	Ericsson, K. A., Krampe, R. T., & Tesch-Römer, C. (1993). The role of deliberate practice in the acquisition of expert performance. <i>Psychological Review</i> , 100(3), 363–406.

THANK YOU

Questions

Full technical documentation, algorithms, evaluation and the complete annotated bibliography are in `docs/Adaptive_Chess_Puzzle_Advisor_Technical_Report.pdf` (37 pp.) and `dissertation_documentation.md`.

```
python web/backend/app.py
pytest tests/ -v      # 128 tests
```

